



Οικονομικό Πανεπιστήμιο Αθηνών, Τμήμα Πληροφορικής Παράλληλος Προγραμματισμός 2025-26

#3 Προγραμματιστική Εργασία (CUDA/OpenCL)

Ανθίπη Φατσέα

AM: 3190209

E-mail: p3190209@aueb.gr

Περιγραφή Συστήματος

Όνομα υπολογιστή	anthippi
Επεξεργαστής	AMD Ryzen AI 7 350
Λειτουργικό Σύστημα	Windows 11 (MSYS2/UCRT64)
Πλήθος πυρήνων	8 Physical Cores / 16 Logical Processors
Μεταφραστής	gcc v15.2.0

Χαρακτηριστικά Κάρτας Γραφικών (GPU)

Μοντέλο GPU	NVIDIA GeForce RTX 5060 Laptop GPU
Compute Capability	12.0
Συνολική Μνήμη (Global Memory)	7.96 GB
Μνήμη Shared ανά Block	48 KB
Μέγιστος Αριθμός Threads ανά Block	1024
Μέγεθος Warp (Warp Size)	32 Threads
Αριθμός Multiprocessors (SMs)	26
Μνήμη Κρυφής Μνήμης (L2 Cache)	32 MB (32768 KB)
Ταυτόχρονη Εκτέλεση Kernels	Ναι

Contents

Περιγραφή Συστήματος	1
Χαρακτηριστικά Κάρτας Γραφικών (GPU)	1
Σειριακή Εκτέλεση στην Κάρτα Γραφικών	2
Μέρος Α	3
Πειραματικά Αποτελέσματα και Μετρήσεις	5
Regular vs Irregular για φόρτο N=1.000.000	8
Μέρος Β	9
Ποσοτική Σύγκριση	9
Επίδραση του Διαχειριστικού Κόστους σε Μικρό Μέγεθος Προβλήματος N = 100.000 ...	10
Τελικό Συμπέρασμα Αναφοράς	11

Σειριακή Εκτέλεση στην Κάρτα Γραφικών

Πριν προχωρήσουμε στην ανάλυση των παράλληλων εκδόσεων του αλγορίθμου, κρίνεται σκόπιμο να οριστεί μια βάση σύγκρισης της απόδοσης. Στην παρούσα αναφορά η σύγκριση θα εστιαστεί στην αρχιτεκτονική της κάρτας γραφικών.

Για το σκοπό αυτό, δημιουργήθηκε ένας ειδικός πυρήνας (gru_serial) ο οποίος κλήθηκε με ακριβώς 1 Block και 1 Thread (<<<1, 1>>>), αναγκάζοντας την κάρτα γραφικών να υπολογίσει το ολοκλήρωμα απολύτως σειριακά.

```
cudaEvent_t start, end;
cudaEventCreate(&start); cudaEventCreate(&end);
cudaEventRecord(start);

kernel_serial<<<1, 1>>>(a, h, N, d_result, irregular);

cudaEventRecord(end);
cudaEventSynchronize(end);

float ms;
cudaEventElapsedTime(&ms, start, end);
total_elapsed += (ms / 1000.0);

cudaEventDestroy(start); cudaEventDestroy(end);
```

Πίνακας 1: Χρόνοι σειριακής εκτέλεσης (1 GPU Thread) ανά μέγεθος προβλήματος

Πλήθος Υποδιαιρέσεων	Χρόνος Regular	Χρόνος Irregular
----------------------	----------------	------------------

1.000	0.002814	0.185288
10.000	0.001735	1.863519
100.000	0.017031	18.575566
1.000.000	0.169489	- (Δεν εκτελέστηκε)

Για τα μεγαλύτερα μεγέθη προβλήματος, η σειριακή εκτέλεση παραλείφθηκε σκοπίμως, καθώς ο υπολογιστικός φόρτος σε συνδυασμό με την χαμηλή συχνότητα του μεμονωμένου πυρήνα της GPU καθυστερούσε υπερβολικά.

Σχολιασμός

Σε αντίθεση με τους πυρήνες μιας CPU, οι πυρήνες της GPU είναι απλούστεροι και χρονισμένοι σε χαμηλότερη συχνότητα. Είναι χαρακτηριστικό ότι για τον ακανόνιστο φόρτο με μόλις $N=100.000$, ένας και μόνο πυρήνας της GPU χρειάστηκε πάνω από 18.5 δευτερόλεπτα για να ολοκληρώσει. Αυτοί οι χρόνοι αποδεικνύουν ότι η αρχιτεκτονική της GPU δεν είναι σχεδιασμένη για σειριακές εκτελέσεις και η πραγματική της δύναμη μπορεί να αναδειχθεί μόνο μέσω του μαζικού παραλληλισμού.

Μέρος Α

Στο πρώτο μέρος της εργασίας, η αριθμητική ολοκλήρωση της συνάρτησης παραλληλοποιήθηκε στο περιβάλλον της κάρτας γραφικών μέσω του προτύπου CUDA, με βασικό στόχο τη μελέτη της επίδρασης που έχει η ιεραρχία μνήμης της GPU στον συνολικό χρόνο εκτέλεσης. Για τον σκοπό αυτό εξετάστηκαν τέσσερις διαφορετικές προσεγγίσεις.

Ξεκινώντας από την απλοϊκή **Naive** (καθώς και την **Global Grid-Stride**), κάθε νήμα αναλαμβάνει τους υπολογισμούς του και προσθέτει το αποτέλεσμα απευθείας στην κεντρική μνήμη χρησιμοποιώντας την οδηγία `atomicAdd` σε κάθε επανάληψη. Αυτή η προσέγγιση δημιουργεί ένα ισχυρό υπολογιστικό εμπόδιο (*bottleneck*) λόγω του συνεχούς αποκλεισμού της καθολικής μνήμης.

Η απόδοση βελτιώνεται αισθητά με τη μετάβαση στη χρήση τοπικών καταχωρητών (**Registers**). Εδώ, κάθε νήμα χρησιμοποιεί μια τοπική μεταβλητή για να αθροίσει τα μερικά του αποτελέσματα και εκτελεί την `atomicAdd` στην Global Memory μόνο μία φορά, στο τέλος της εκτέλεσής του.

```

// Naive
// kathe nima grafete katefthia sto global memory
__global__ void kernel_naive(double a, double h, int N, double *d_sum, bool irregular) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        double x1 = a + i*h;
        double x2 = a + (i+1)*h;
        atomicAdd(d_sum, (get_f(x1, irregular) + get_f(x2, irregular)) * h / 2.0);
    }
}

// grid-stride me registers
// kathe nima xrisimopoei dikotou local_sum
__global__ void kernel_registers(double a, double h, int N, double *d_sum, bool irregular) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    double local_sum = 0.0;
    while (i < N) {
        double x1 = a + i*h;
        double x2 = a + (i+1)*h;
        local_sum += (get_f(x1, irregular) + get_f(x2, irregular)) * h / 2.0;
        i += blockDim.x * gridDim.x;
    }
    atomicAdd(d_sum, local_sum);
}

```

Τέλος, στη βέλτιστη προσέγγιση **Shared Memory**, αξιοποιείται η ταχύτατη κοινόχρηστη μνήμη επιπέδου Block. Τα νήματα κάθε ομάδας αποθηκεύουν τα αποτελέσματά τους σε έναν δυναμικά δεσμευμένο κοινό πίνακα sdata και συγχρονίζονται. Στη συνέχεια, μέσω ενός αλγορίθμου παράλληλης αναγωγής (Parallel Tree Reduction), ένα μόνο νήμα (το tid == 0) αναλαμβάνει να γράψει το συνολικό άθροισμα όλου του Block στην Global Memory, ελαχιστοποιώντας πλήρως τις προσβάσεις σε αυτή.

```

// optimal me shared memory
// ta nimata kathe block exoun mia kini metagliti opou ekei athizoun ola tis times tous
// kai sto telos ena nima epistrefei to apotelesma
__global__ void kernel_shared(double a, double h, int N, double *d_sum, bool irregular)
extern __shared__ double sdata[]; // i kini timi mpainei sto sdata
int tid = threadIdx.x;
int i = blockIdx.x * blockDim.x + threadIdx.x;

double local_sum = 0.0;
while (i < N) {
    double x1 = a + i*h;
    double x2 = a + (i+1)*h;
    local_sum += (get_f(x1, irregular) + get_f(x2, irregular)) * h / 2.0;
    i += blockDim.x * gridDim.x;
}
sdata[tid] = local_sum;
__syncthreads(); // perimenoun ta nimata na teliosoun ola

// athisma apotelesmaton
for (int s = blockDim.x / 2; s > 0; s >>= 1) {
    if (tid < s) sdata[tid] += sdata[tid + s];
    __syncthreads();
}

// to arxiko nima epistrefei to apotelesma
if (tid == 0) atomicAdd(d_sum, sdata[0]);
}

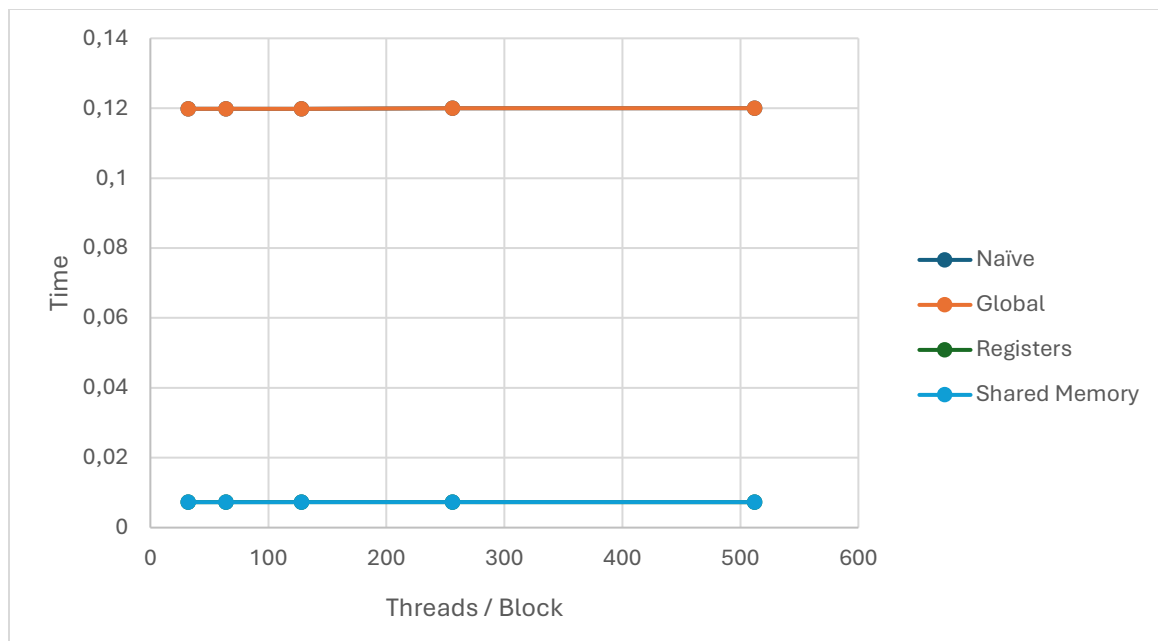
```

Πειραματικά Αποτελέσματα και Μετρήσεις

Οι μετρήσεις πραγματοποιήθηκαν διατηρώντας το μέγεθος του προβλήματος σταθερό στα $N = 100.000.000$ διαστήματα, μεταβάλλοντας τον αριθμό των νημάτων ανά Block .

Πίνακας : Χρόνοι εκτέλεσης CUDA για $N = 100.000.000$ Regular Mode

Threads / Block	Naive	Global	Registers	Shared Memory
32	0.119842	0.119852	0.007266	0.007281
64	0.119848	0.119851	0.007271	0.007280
128	0.119850	0.119863	0.007280	0.007281
256	0.119988	0.119992	0.007287	0.007305
512	0.119984	0.119988	0.007266	0.007303



Σχόλια και παρατηρήσεις

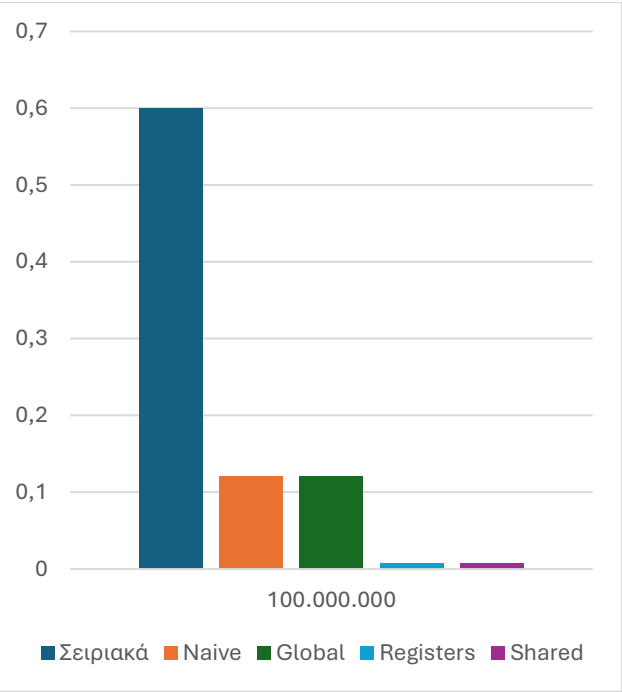
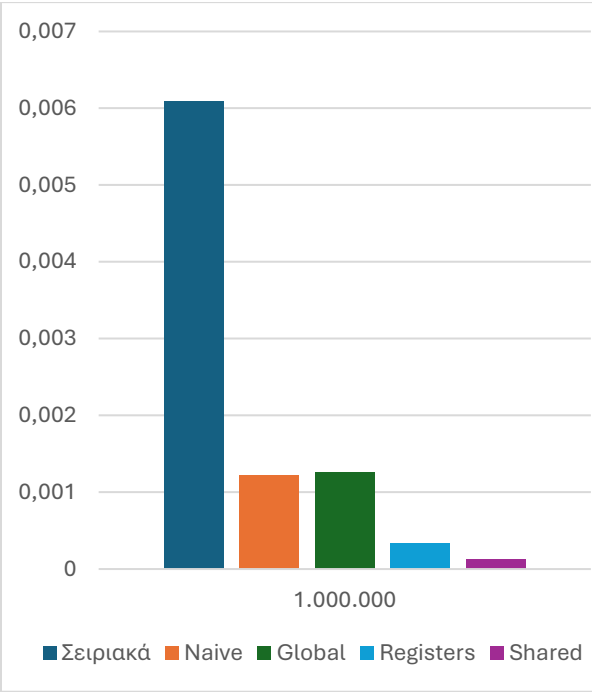
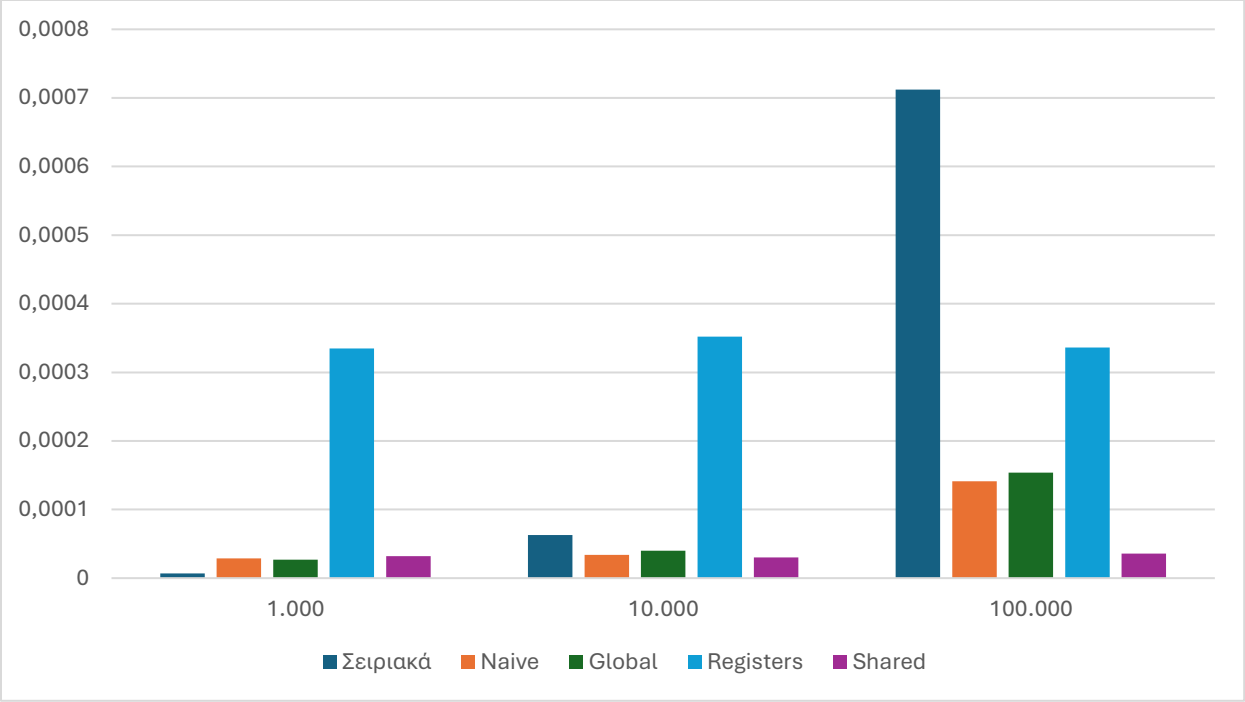
Οι προσεγγίσεις **Naive** και **Global** παρουσιάζουν πανομοιότυπους και συγκριτικά πολύ αργούς χρόνους. Αυτό οφείλεται στο γεγονός ότι χιλιάδες νήματα προσπαθούν ταυτόχρονα να προσπελάσουν και να τροποποιήσουν την ίδια μεταβλητή `d_sum` στην αργή Global Memory. Ο συνεχής αποκλεισμός μέσω της `atomicAdd` προκαλεί τεράστιο μπουτιλιάρισμα, μετατρέποντας το πρόβλημα σε *Memory-Bound*.

Αντίθετα ελαχιστοποιώντας τις προσβάσεις στην παγκόσμια μνήμη μία ανά νήμα στους Registers και μόλις μία ανά Block στη Shared, η GPU μπόρεσε να αξιοποιήσει στο έπακρο την υπολογιστική της ισχύ, παρακάμπτοντας το "Memory Wall".

Επίσης αξιοσημείωτο είναι το γεγονός ότι ο αριθμός των νημάτων ανά Block δεν επηρέασε σχεδόν καθόλου τον χρόνο εκτέλεσης στο μέγιστο μέγεθος προβλήματος. Αυτό αποδεικνύει ότι για $N = 10^8$, η GPU τροφοδοτείται με επαρκή φόρτο εργασίας ώστε να κρατάει όλους τους Streaming Multiprocessors απασχολημένους, κρύβοντας απόλυτα τον χρόνο λανθάνουσας μνήμης. Οι χρόνοι ~ 0.007 sec αποτελούν πρακτικά το μέγιστο όριο απόδοσης της κάρτας γραφικών για τον συγκεκριμένο αλγόριθμο.

Πίνακας 2: Χρόνοι Εκτέλεσης με σταθερό Block Size (256 Threads) μεταβάλλοντας το N

Μέγεθος N	Σειριακά	Naive	Global	Registers	Shared
1.000	0.000007	0.000029	0.000027	0.000335	0.000032
10.000	0.000063	0.000034	0.000040	0.000352	0.000030
100.000	0.000712	0.000141	0.000154	0.000336	0.000036
1.000.000	0.006085	0.001224	0.001255	0.000335	0.000124
100.000.000	0.600340	0.119988	0.119992	0.007287	0.007305



Σχόλια και παρατηρήσεις

Παρατηρείται ότι οι χρόνοι για τους πυρήνες Naive, Global και Shared είναι σχεδόν ταυτόσημοι. Ο χρόνος που καταγράφεται αφορά κυρίως τον χρόνο που χρειάζεται η CPU για να στείλει την εντολή στην GPU, να δεσμευτούν οι πόροι και να ξεκινήσει η εκτέλεση. Το διαχειριστικό αυτό κόστος επικαλύπτει οποιοδήποτε πλεονέκτημα ταχύτητας προσφέρει η ιεραρχία μνήμης.

Η πραγματική αξία της Shared Memory και των Registers αρχίζει να διαφαίνεται καθαρά από το $N = 1.000.000$ και έπειτα, όπου ο όγκος των δεδομένων προκαλεί ασφυξία στην Global Memory για τις μεθόδους Naive και Global. Στο μέγιστο μέγεθος, οι βελτιστοποιημένες εκδόσεις αναδεικνύουν την υπεροχή τους.

Regular vs Irregular για φόρτο $N=1.000.000$

Για την πληρέστερη κατανόηση της συμπεριφοράς της κάρτας γραφικών, παραθέτουμε τη σύγκριση των χρόνων εκτέλεσης μεταξύ της ομαλής και της συνάρτησης με την μετατροπή για το μέγεθος $N=1.000.000$.

Πίνακας: Σύγκριση Χρόνων Εκτέλεσης (Shared)

Threads / Block	Regular (Shared)	Irregular (Shared)
32	0.000093	0.057853
64	0.000090	0.057665
128	0.000107	0.057812
256	0.000124	0.058654
512	0.000125	0.057492

Σχολιασμός

Ενώ στην περίπτωση της Regular συνάρτησης η πλήρης αξιοποίηση της Shared Memory και της κρυφής μνήμης διατηρεί τους χρόνους εκτέλεσης σε επίπεδο μικροδευτερολέπτων, στην Irregular συνάρτηση οι χρόνοι εκτοξεύονται στα ~ 0.057 δευτερόλεπτα, καθώς το πρόβλημα μετατρέπεται από Memory-Bound σε Compute-Bound λόγω του φαινομένου Warp Divergence. Συγκεκριμένα, η ακανόνιστη φύση της συνάρτησης επιβάλλει διαφορετικό αριθμό επαναλήψεων στα νήματα του ίδιου Warp, αναγκάζοντας όσα ολοκληρώνουν γρήγορα να παραμένουν σε αδράνεια μέχρι να τελειώσουν τα "βαριά" νήματα, γεγονός που εξαιλεί τα οφέλη της Shared Memory και μετατοπίζει την καθυστέρηση από την ανάκτηση δεδομένων στην εκτέλεση εντολών. Η σταθερότητα των χρόνων αυτών ανεξαρτήτως του αριθμού των νημάτων ανά Block επιβεβαιώνει ότι η GPU έχει φτάσει στο όριο του Warp Divergence, όπου η σειριακή εκτέλεση εντός των Warps καθίσταται ο κυρίαρχος περιοριστικός παράγοντας της απόδοσης.

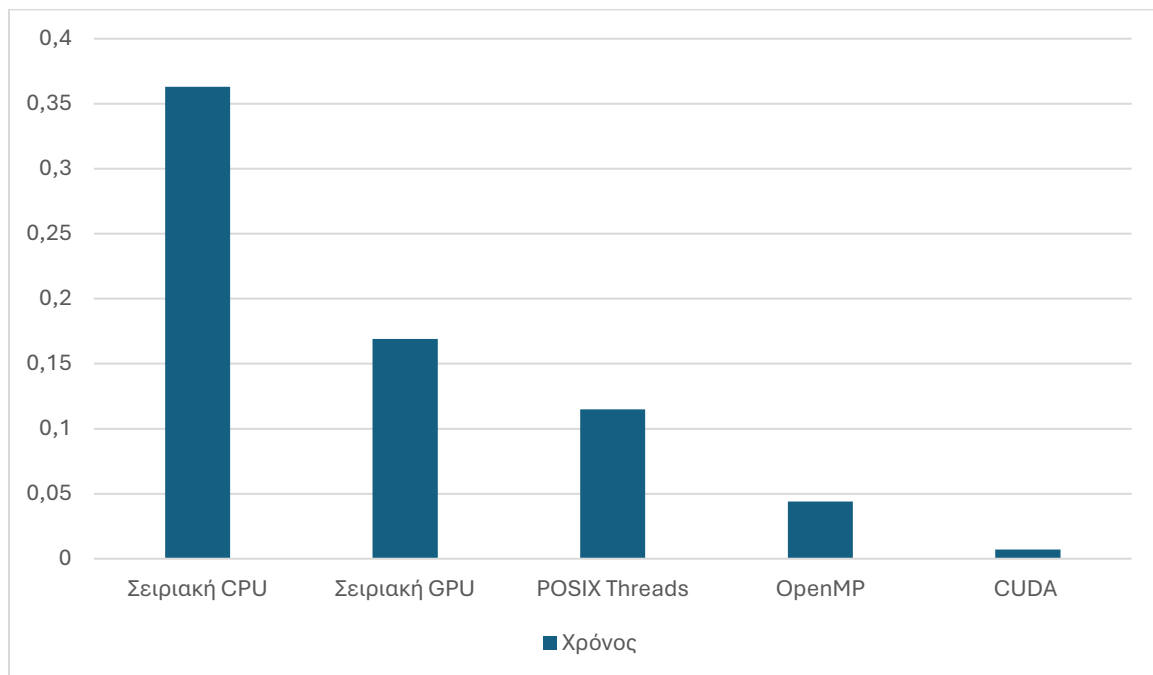
Μέρος Β

Στο τελικό στάδιο της μελέτης, πραγματοποιείται η συγκριτική αξιολόγηση των τριών μοντέλων προγραμματισμού που εξετάστηκαν (POSIX Threads, OpenMP, CUDA), με σκοπό την ανάδειξη της αποδοτικότητας κάθε μεθόδου στην επίλυση του προβλήματος της αριθμητικής ολοκλήρωσης.

Ποσοτική Σύγκριση

Πίνακας: Σύγκριση Καλύτερων Υλοποιήσεων (N = 100.000.000)

Μοντέλο	Καλύτερη Υλοποίηση	Χρόνος
Σειριακή CPU		0.363
Σειριακή GPU		0.169
POSIX Threads	A2 Τοπική συσσώρευση	0.115
OpenMP	Reduction/ No Reduction	0.044
CUDA	Registers	0.007



Σχολιασμός και Συμπεράσματα

Από την ποσοτική σύγκριση των βέλτιστων υλοποιήσεων για μέγεθος προβλήματος N = 100.000.000, προκύπτει η σαφής υπεροχή της αρχιτεκτονικής GPU. Η υλοποίηση CUDA (με

χρήση Registers/Shared Memory) επιτυγχάνει χρόνο 0.007 sec, όντας κατά 6x ταχύτερη από το OpenMP και πάνω από 16x ταχύτερη από την καλύτερη προσέγγιση των POSIX Threads. Αυτή η τεράστια επιτάχυνση αναδεικνύει τις θεμελιώδεις διαφορές στον τρόπο διαμοιρασμού της δουλειάς. Τα CPU-based μοντέλα περιορίζονται εγγενώς από το hardware τους φυσικούς πυρήνες του συστήματος, αναθέτοντας βαρύ υπολογιστικό φόρτο σε λίγα, ισχυρά νήματα. Αντίθετα, η CUDA αξιοποιεί τον μαζικό παραλληλισμό, διασπώντας το πρόβλημα σε χιλιάδες ελαφριά νήματα που εκτελούνται ταυτόχρονα στους Streaming Multiprocessors.

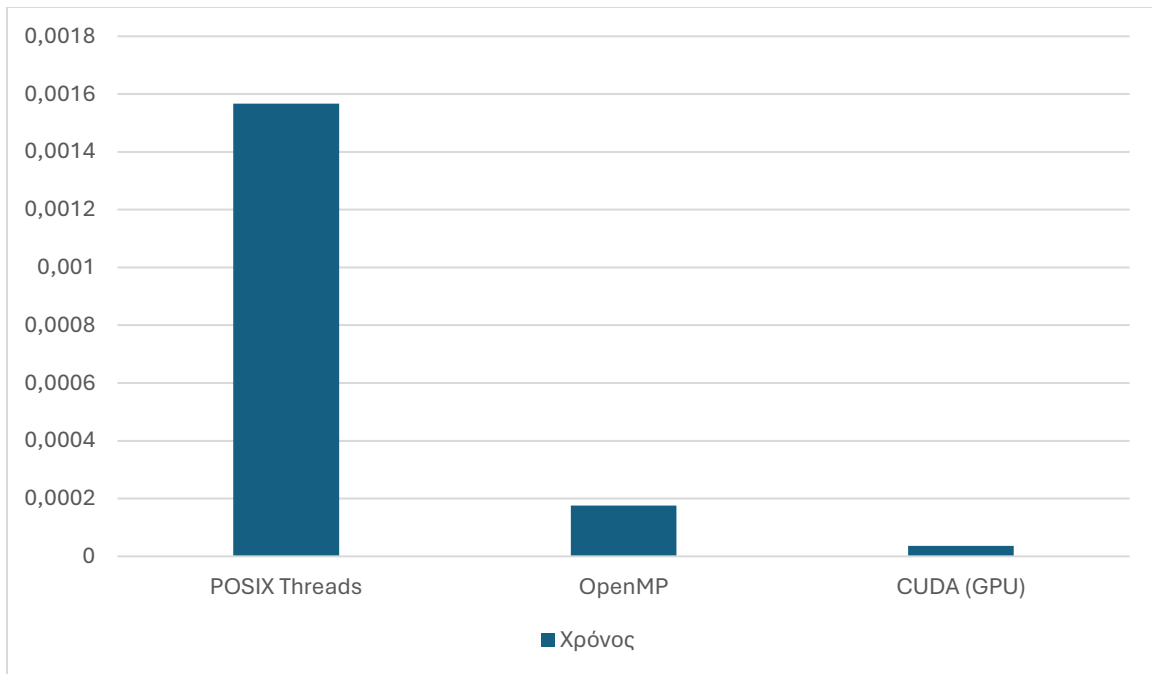
Αναλύοντας τα πλεονεκτήματα και τις αδυναμίες κάθε μεθόδου, τα POSIX Threads προσφέρουν απόλυτο έλεγχο χαμηλού επιπέδου, αλλά απαιτούν πολύπλοκο κώδικα και είναι επιρρεπή σε υψηλό διαχειριστικό κόστος, το οποίο χρειάστηκε να παρακαμφθεί χειροκίνητα. Το OpenMP αποτελεί την ιδανική λύση για συστήματα πολυπύρηνων επεξεργαστών, καθώς με ελάχιστες οδηγίες εξαλείφει αυτόματα το synchronization overhead και απλοποιεί τη δημιουργία εργασιών. Τέλος, η CUDA παρουσιάζει το μειονέκτημα της υψηλής προγραμματιστικής δυσκολίας, απαιτώντας ρητή διαχείριση της μνήμης της GPU και αντιμετωπίζοντας δυσκολίες στον ακανόνιστο φόρτο λόγω του Warp Divergence. Ωστόσο, μέσω τεχνικών όπως το Parallel Tree Reduction στη Shared Memory, η CUDA μειώνει το κόστος συγχρονισμού αποφεύγοντας την αργή Global Memory, αποδεικνύοντας ότι είναι το απόλυτο εργαλείο για προβλήματα υψηλού όγκου δεδομένων.

Επίδραση του Διαχειριστικού Κόστους σε Μικρό Μέγεθος Προβλήματος $N = 100.000$

Για να κατανοηθεί πλήρως η συμπεριφορά των τριών μοντέλων προγραμματισμού, αξιολογούνται συγκριτικά και για ένα σημαντικά μικρότερο μέγεθος προβλήματος ($N = 100.000$) στην ομαλή συνάρτηση (Regular Mode).

Πίνακας: Σύγκριση Καλύτερων Υλοποιήσεων ($N = 100.000$, Regular Mode)

Μοντέλο	Καλύτερη Υλοποίηση	Χρόνος
POSIX Threads	A2 Τοπική Συσσώρευση (1 νήμα)	0.001567
OpenMP	Reduction (8 νήματα)	0.000176
CUDA (GPU)	Shared Memory (256 threads)	0.000036



Σχολιασμός και Συμπεράσματα

Το OpenMP, παρότι βασίζεται επίσης στην CPU, διαχειρίζεται τα νήματα πολύ πιο αποδοτικά μέσω του εσωτερικού μηχανισμού του compiler. Σε αντίθεση με τα Pthreads, εμφανίζει σημαντική επιτάχυνση, καθιστώντας το πολύ πιο ανθεκτικό σε μικρά grain sizes.

Η κάρτα γραφικών παραμένει η ταχύτερη, όμως η διαφορά της από τη CPU δεν είναι πλέον τόσο χαώδης όσο στο μέγιστο μέγεθος $N=100.000.000$. Σε αυτή την κλίμακα, η απόδοση της GPU "φρενάρει" από το Kernel Launch Overhead. Ο χρόνος που καταγράφεται αφορά κατά κύριο λόγο τον χρόνο που χρειάζεται η CPU για να στείλει την εντολή στην GPU μέσω του διαύλου PCIe και να ξεκινήσει η εκτέλεση των πόρων.

Τελικό Συμπέρασμα Αναφοράς

Εν κατακλείδι, η επιλογή του βέλτιστου προγραμματιστικού μοντέλου δεν καθορίζεται από μια απόλυτη υπεροχή, αλλά από την πλήρη κατανόηση των χαρακτηριστικών του προβλήματος Data size, Load Balancing και την εναρμόνισή τους με την αρχιτεκτονική του διαθέσιμου υλικού.